

Guide complet de deploiement (Gratuit) - BOUANE O DOUMANTANG

Ce guide decrit pas a pas comment mettre en ligne le site (frontend Vite + backend Django) en utilisant des services gratuits (Vercel, Railway/Fly.io/Railway, Supabase, Cloudinary, Cloudflare). Il fournit les commandes, variables d'environnement et verifications a effectuer. Suivez les sections dans l'ordre.

--

1. Choix de la stack gratuite recommandee

- * Frontend (static) : Vercel (free) ou Netlify
- * Backend (Django) : Railway / Fly.io / Render free tier (Railway recommande pour facilite)
- * Base de donnees : Supabase (Postgres free) ou Railway Postgres free credits
- * Stockage medias (uploads) : Cloudinary (free) ou Supabase Storage
- * DNS & TLS : Cloudflare (free)
- * Remarque : Les offres free ont des limitations (mise en veille, quotas). Pour production stable, prevoir un plan payant.

2. Prerequis

- * Compte GitHub (ou Git provider) contenant le repo.
- * Comptes: Vercel, Railway (ou Fly.io), Supabase, Cloudinary (optionnel), Cloudflare (optionnel).
- * Node.js et npm installes localement pour builder le frontend.
- * Python 3.10+/venv local pour tests.

3. Preparer le code localement (verifications rapides)

1. Frontend

2. Ouvrir frontend/package.json : verifier script build (Vite). Exemple:

```
json
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview --port 5173"
}
```

3.

Tester build local :

```
bash
cd frontend
npm install
npm run build
npx serve dist # ou npm i -g serve
```

4.

Backend

5. Verifier backend/requirements.txt contient gunicorn, psycopg2-binary (ou psycopg[binary]), django-environ si utilise.

6. Key files to check: backend/bouano/settings.py, backend/manage.py, backend/bouano/wsgi.py.

7. Tester localement:

```
bash
cd backend
python -m venv .venv
.venv/Scripts/activate # Windows
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

frontend/package.json

build

json

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview --port 5173"
}
```

Tester build local :

```
bash
cd frontend
npm install
npm run build
npx serve dist # ou npm i -g serve
```

bash

```
cd frontend
npm install
npm run build
npx serve dist # ou npm i -g serve
```

Backend

backend/requirements.txt

unicorn

psycopg2-binary

psycopg[binary]

django-environ

backend/bouano/settings.py

backend/manage.py

backend/bouano/wsgi.py

bash

```
cd backend
python -m venv .venv
.venv/Scripts/activate # Windows
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

4. Variables d'environnement a preparer

* DJANGO_SECRET_KEY : cle secrete Django

* DEBUG=false

- * ALLOWED_HOSTS : backend host(s) (ex: backend-xxxxx.up.railway.app)
- * DATABASE_URL : URL Postgres (Supabase/Railway)
- * TARAMONEY_API_KEY : (si utilise)
- * TARAMONEY_BUSINESS_ID
- * TARAMONEY_BASE_URL
- * TARAMONEY_DEFAULT_WEBHOOK : URL publique /api/payments/taramoney/webhook/
- * CLOUDINARY_URL or SUPABASE_STORAGE_* if using external media storage
- * VITE_API_BASE_URL : (pour frontend build env) ex: https://<backend-host>/api

DJANGO_SECRET_KEY

DEBUG=false

ALLOWED_HOSTS

backend-xxxxx.up.railway.app

DATABASE_URL

TARAMONEY_API_KEY

TARAMONEY_BUSINESS_ID

TARAMONEY_BASE_URL

TARAMONEY_DEFAULT_WEBHOOK

/api/payments/taramoney/webhook/

CLOUDINARY_URL

SUPABASE_STORAGE_*

VITE_API_BASE_URL

https://<backend-host>/api

Note: Never commit secret values to Git.

5. Deploiement Frontend (Vercel)

1. Connecter le repo GitHub a Vercel.
2. Project settings:
3. Framework: "Vite"
4. Build command: npm run build
5. Output directory: dist
6. Add Environment Variable (Production):
7. VITE_API_BASE_URL = https://<backend-host>/api
8. Deployer: Vercel build & publish. Copiez l'URL publique fournie.

npm run build

dist

VITE_API_BASE_URL

https://<backend-host>/api

6. Deploiement Backend (Railway guide)

1. Créer un nouveau projet sur Railway, connecter le repo backend.
2. Définir le service web et la commande de démarrage. Exemple Procfile (ajouter au repo):
Procfile
web: gunicorn bouano.wsgi --bind 0.0.0.0:\$PORT --workers 2
3. Ajouter env vars dans Railway (Settings Variables) listées en section 4.
4. Attacher une base de données (Supabase or Railway Postgres) et définir DATABASE_URL.

5. Deployer: Railway executera pip install et demarrera le service.

Procfile

Procfile

```
web: gunicorn bouano.wsgi --bind 0.0.0.0:$PORT --workers 2
```

DATABASE_URL

pip install

Alternative: utiliser Dockerfile si vous preferez deployer en conteneur.

7. Base de donnees (Supabase)

1. Creez un projet Supabase, recuperez les credetials (host, port, db, user, password).

2. Recuperez DATABASE_URL au format:

```
postgres://user:password@host:port/dbname
```

3. Executer migrations sur le serveur (via Railway console ou CICD deploy step):

```
bash
```

```
python manage.py migrate
```

```
python manage.py loaddata initial_data.json # facultatif
```

```
python manage.py createsuperuser
```

DATABASE_URL

```
postgres://user:password@host:port/dbname
```

```
bash
```

```
python manage.py migrate
```

```
python manage.py loaddata initial_data.json # facultatif
```

```
python manage.py createsuperuser
```

8. Stockage medias

* Option Cloudinary (simple): installez django-cloudinary-storage et configurez DEFAULT_FILE_STORAGE = 'cloudinary_storage.storage.MediaCloudinaryStorage' et CLOUDINARY_URL env var.

* Option Supabase Storage: configurez django-storages compatible ou upload via API directement.

* Mettez a jour backend/bouano/settings.py pour pointer vers la solution choisie.

```
django-cloudinary-storage
```

```
DEFAULT_FILE_STORAGE = 'cloudinary_storage.storage.MediaCloudinaryStorage'
```

```
CLOUDINARY_URL
```

```
django-storages
```

```
backend/bouano/settings.py
```

9. Webhooks Taramoney

* Taramoney necessite une URL publique stable pour le webhook. Configurez TARAMONEY_DEFAULT_WEBHOOK a <https://<backend-host>/api/payments/taramoney/webhook/>.

* Quelques providers free (Railway) fournissent des URLs stables mais peuvent changer si vous supprimez le projet.

```
TARAMONEY_DEFAULT_WEBHOOK
```

```
https://<backend-host>/api/payments/taramoney/webhook/
```

10. Domaine & DNS (Cloudflare)

1. Acheter un domaine si besoin.

2. Ajouter le domaine sur Vercel (frontend) et Railway (backend) si vous souhaitez custom domains pour les

deux.

3. Dans Cloudflare, creez des CNAME ou A records selon instructions du provider. Activez le proxy Cloudflare (orange cloud) pour TLS et CDN.

CNAME

A

11. collectstatic (Django static)

* Configurer STATIC_ROOT et appeler python manage.py collectstatic pendant le deploiement. Uploader les fichiers statiques sur S3/Cloudflare/CDN ou servez-les via Vercel static site if you copy them into frontend build.

STATIC_ROOT

python manage.py collectstatic

12. Verifications post-deploy

- * Ouvrez l'URL frontend: verifier pages, images, liens.
- * Tester endpoint backend: `https://<backend-host>/api/health/` ou equivalent.
- * Creer une contribution test, lancer paiement (si Taramoney sandbox disponible).
- * Verifier upload d'images et webhook de paiement.

`https://<backend-host>/api/health/`

13. Monitoring & backups gratuits

- * UptimeRobot (free) pour checks.
- * Supabase snapshots / Railway daily backups (free tier limited).

14. Checklist finale (a cocher avant mise en production)

- * ☐ Env vars configurees et secretes stockees.
- * ☐ Base de donnees en place et migrations appliquees.
- * ☐ collectstatic execute et medias accessibles.
- * ☐ Frontend VITE_API_BASE_URL mis a jour.
- * ☐ Webhook Taramoney configure et teste.
- * ☐ Domaine et TLS configures.
- * ☐ Tests manuels du flux de paiement effectues.

collectstatic

VITE_API_BASE_URL

--

Instructions pour generer un PDF (local)

Option A - si tu as pandoc installe:

```
```bash
```

```
pandoc
```

### depuis la racine du projet

```
pandoc DEPLOYMENT_GUIDE.md -o DEPLOYMENT_GUIDE.pdf --pdf-engine=xelatex
```

```
```
```

Option B - via navigateur (simple):

1. Ouvre DEPLOYMENT_GUIDE.md dans VS Code et choisis "Open Preview".
2. Dans la preview, Ctrl+P (Print) ou Export to PDF via l'extension, puis enregistre en PDF.

DEPLOYMENT_GUIDE.md

Ctrl+P

Export to PDF

Option C - convertir HTML -> PDF:

```bash

## generer HTML depuis markdown

pandoc DEPLOYMENT\_GUIDE.md -o DEPLOYMENT\_GUIDE.html

## ouvrir dans Chrome et imprimer en PDF

```

--

Si tu veux, je peux:

- ajouter un Procfile et un Dockerfile au repo,
- creer un fichier deployment_env_example listant toutes les env vars pretes a remplir,
- ou generer directement le PDF si tu souhaites que j'essaie un convertisseur cote serveur (mais il faudra peut-etre installer pandoc localement).

Procfile

Dockerfile

deployment_env_example

pandoc

Dis-moi quelles actions tu veux que j'execute maintenant (ajouter Procfile, Dockerfile, deployment_env_example, ou autre).

Procfile

Dockerfile

deployment_env_example